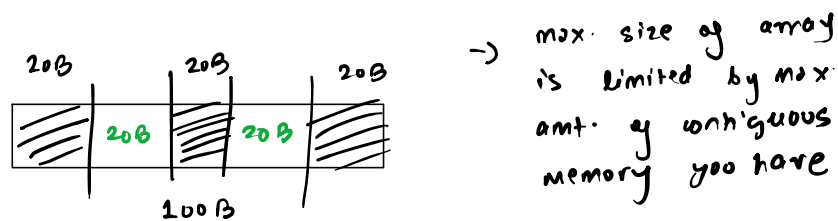
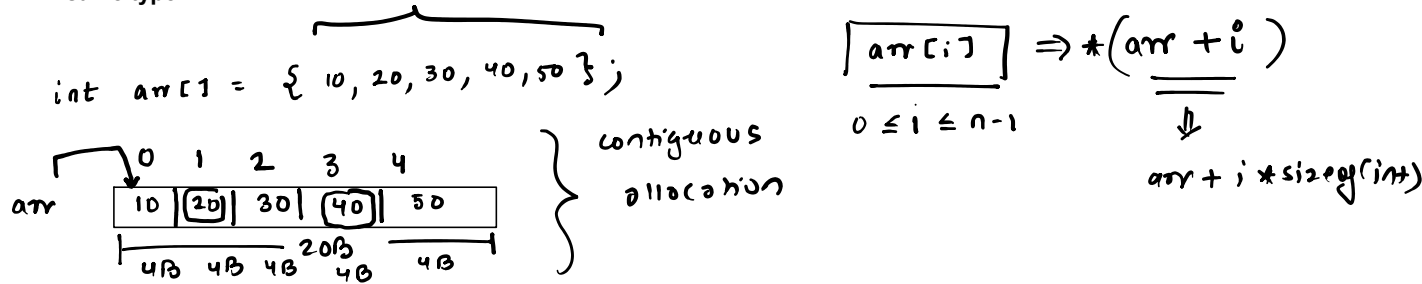


Introduction to Linked List

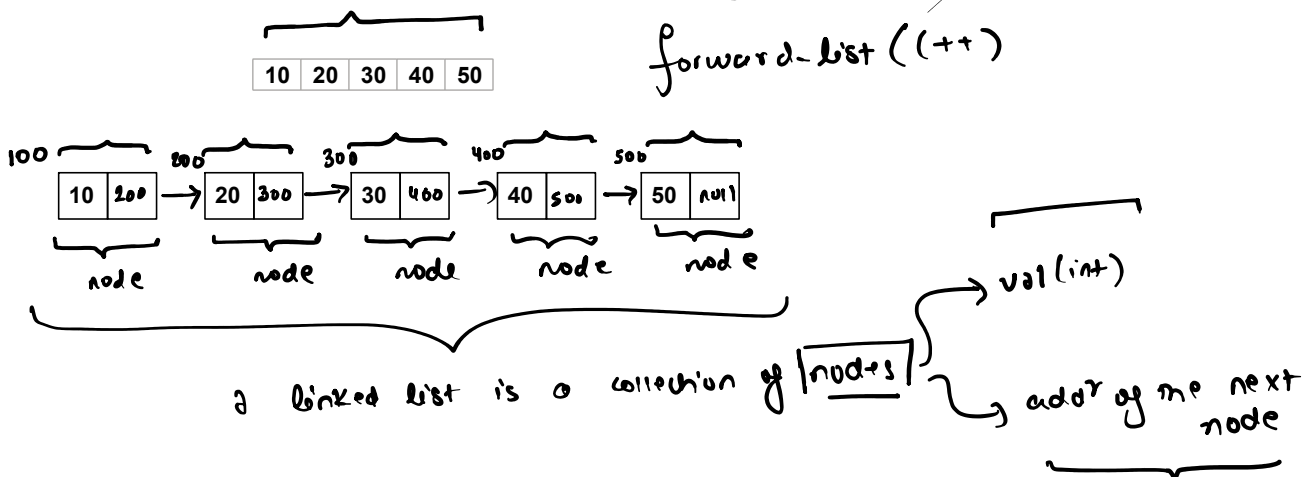
A **Linked List** is a **linear** data structure that is used to store a **sequence** of values of the **same type**.



(vector)
↳ resize op. in dynamic array is expensive
however, in a linked list it is a very cheap op.

Visualizing a Linked List

Singly linked list
forward-list (++)



```

class ListNode {
public :
    int val;
    ListNode* next;

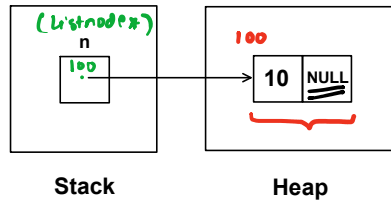
    ListNode(int val) {
        this->val = val;
        this->next = NULL;
    }
};

```

```

int main() {
    ListNode* n = new ListNode(10);
    return 0;
}

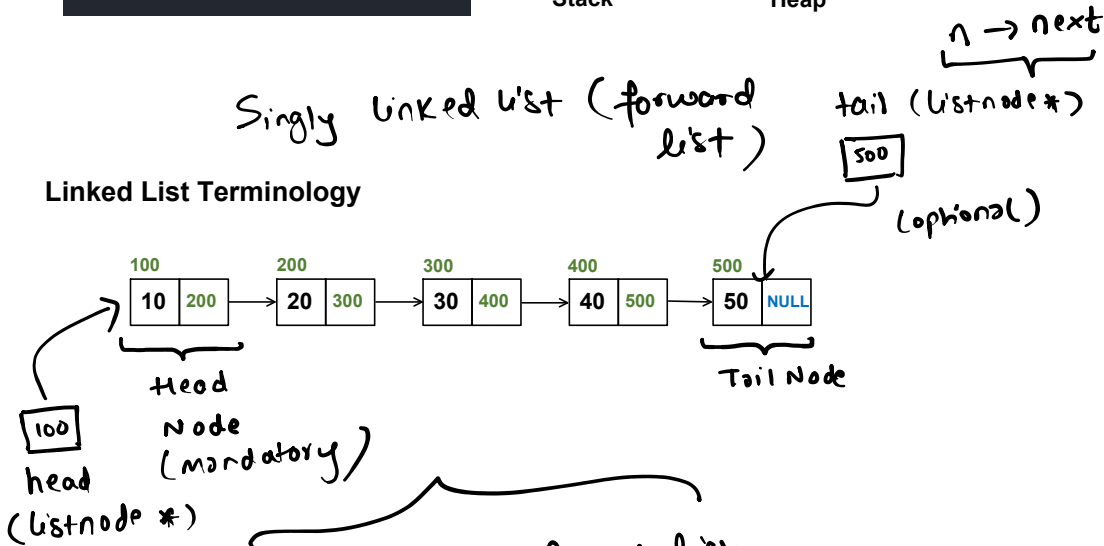
```



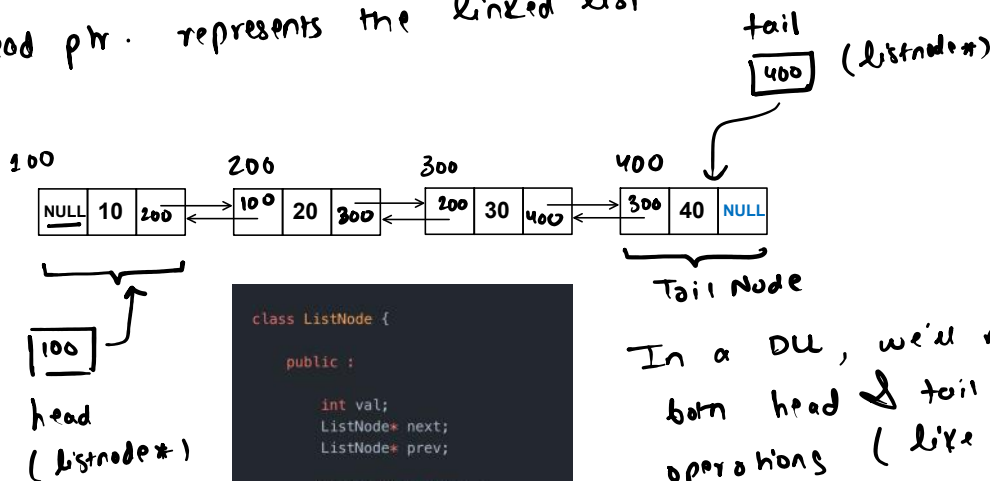
$(*n).val$
 or
 $n \rightarrow val$

Singly Linked List (forward list)

Linked List Terminology



head ptr. represents the linked list



```

class ListNode {
public :
    int val;
    ListNode* next;
    ListNode* prev;

    ListNode(int val) {
        this->val = val;
        this->next = NULL;
        this->prev = NULL;
    }
};

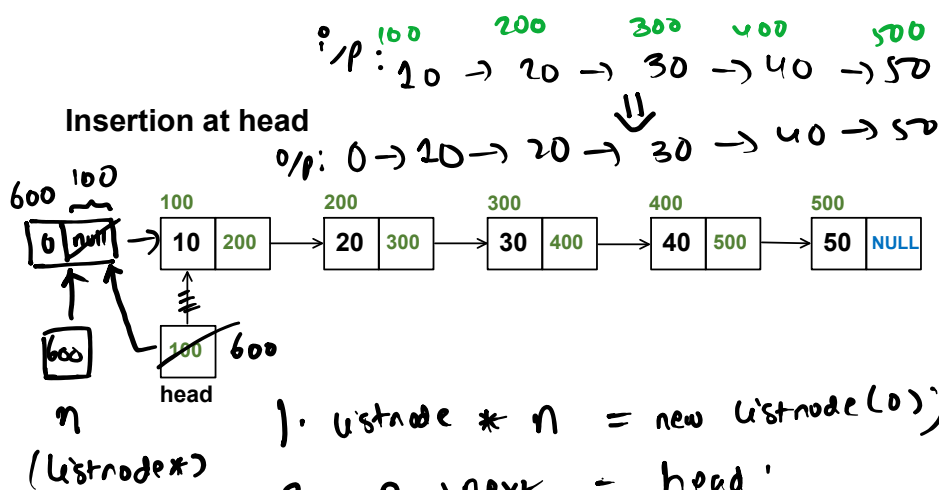
```

In a DLL, we'd maintain both head & tail ptr $\therefore$ operations (like insert/delete) are very efficient on both ends

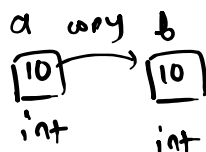
Linked List Insertion

A node in a linked list can be inserted in the following ways -

- ⇒ ○ at the **head** i.e. at its beginning
- at the **tail** i.e. at its end
- at a **random** position i.e. at a given index



int a = 10;
 int b = a;

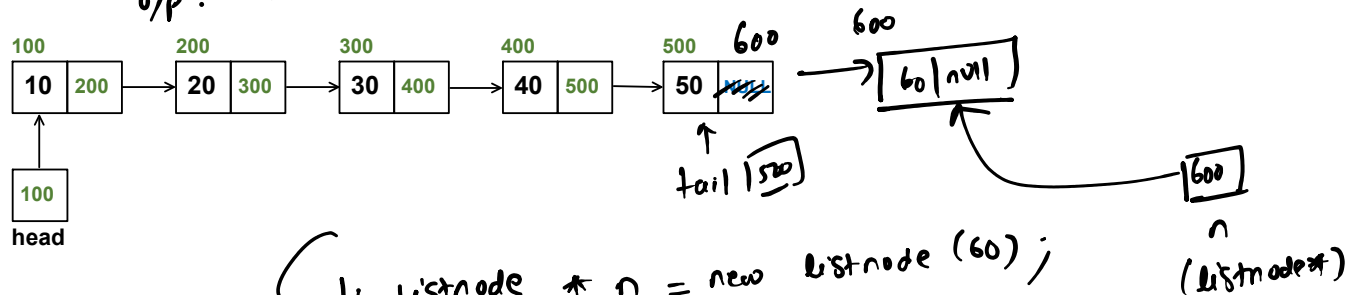


1. `ListNode * n = new ListNode(0);`
 2. `n->next = head;`
 3. `head = n;`

time: $O(1)$

Insertion at tail

i/p: 10 → 20 → 30 → 40 → 50
 o/p: 10 → 20 → 30 → 40 → 50 → 60

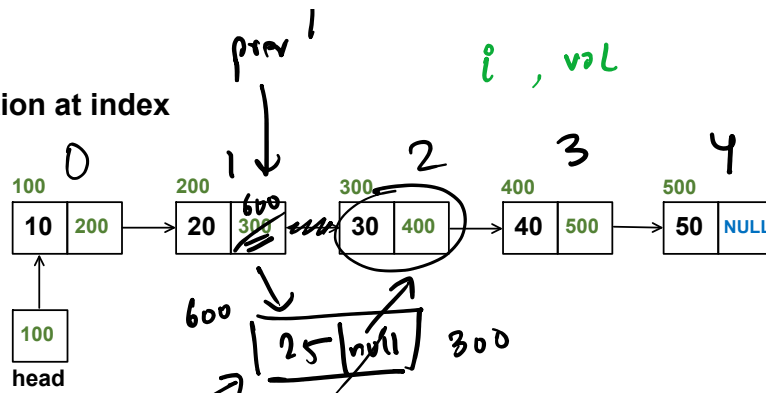


1. `ListNode * n = new ListNode(60);`
 2. `ListNode * tail = getTail(head);`
 3. `tail->next = n;`

prev
 i, val

Insertion at index

Insertion at index



1. `listnode * n = new listnode(val);`
2. `listnode * prev = ...;`
3. `n->next = prev->next;`
4. `prev->next = n;`

Linked List Deletion

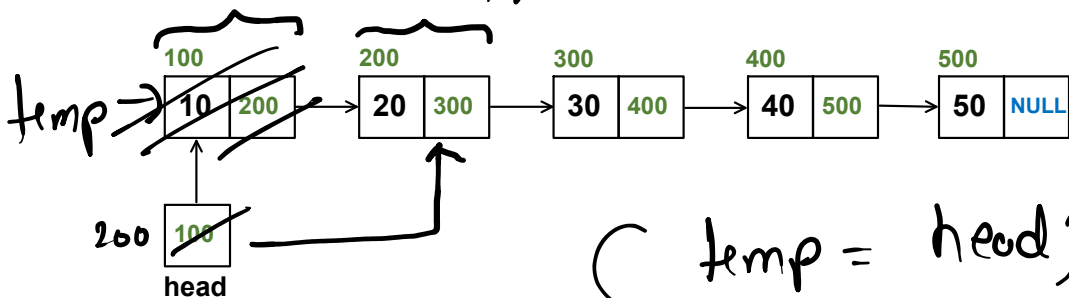
A node in a linked list can be deleted in the following ways -

- from the **head** i.e. from its beginning
- from the **tail** i.e. from its end
- from a **random** position i.e. from a given index

i/p : 10 → 20 → 30 → 40 → 50

o/p : 20 → 30 → 40 → 50

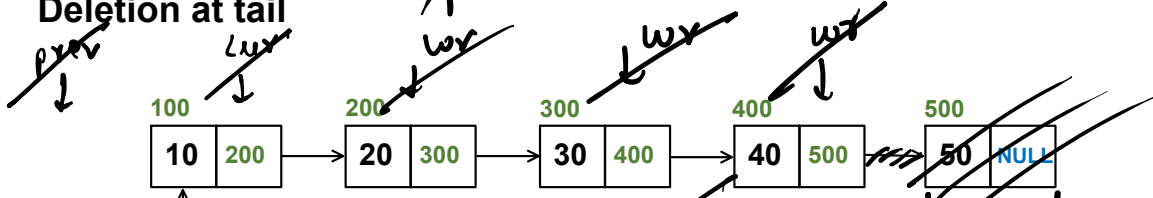
Deletion at head



$O(1)$

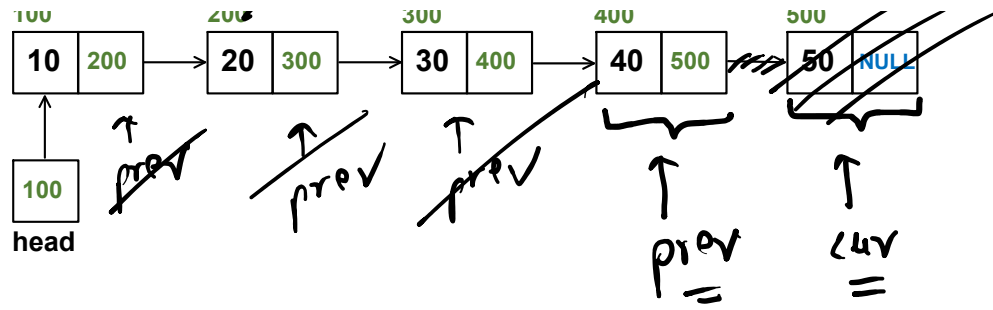
temp = head;
head = head → next
delete temp;

Deletion at tail



time: $O(n)$

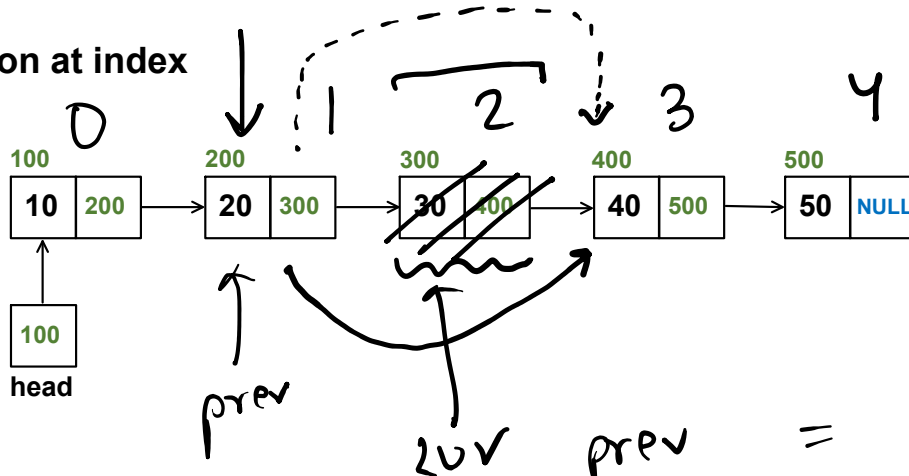
space: $O(1)$



space: O(1)

$prev \rightarrow next = null;$
delete wr;

Deletion at index



$i = 2$

$prev = getNode(i-1)$

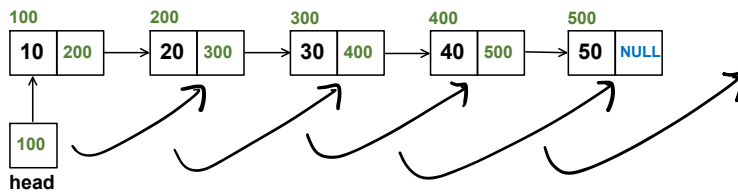
$cur = prev \rightarrow next;$

$prev \rightarrow next = cur \rightarrow next$
delete wr;

Linked List Length

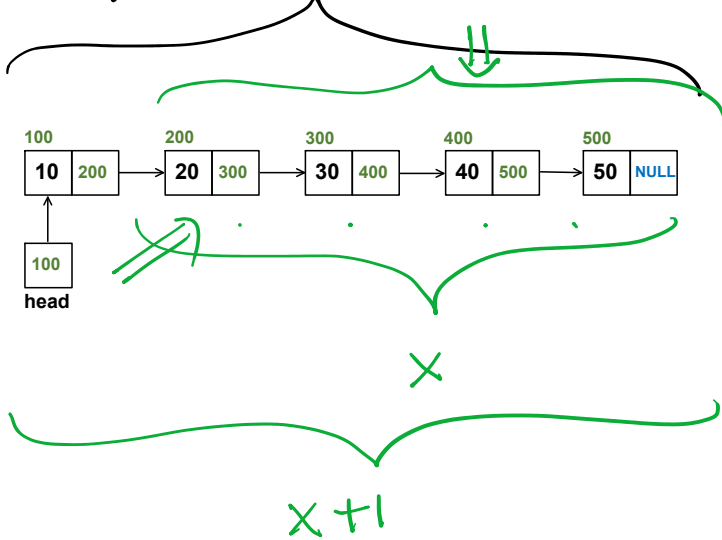
$n = 5$

The length of the a linked list is defined as the **no. of nodes** present in the linked list.



$len = 0/1/2/3/4/5$

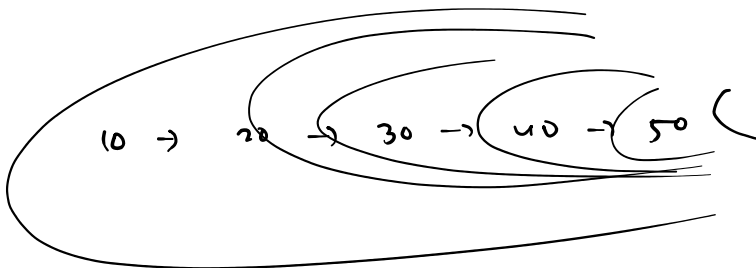
find the len of the LL



$f(head) \{$

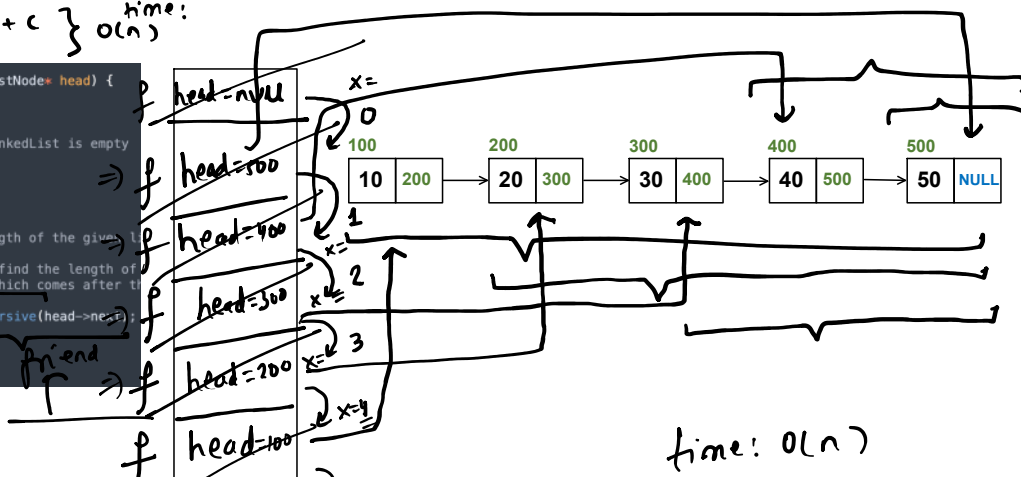
$\text{if}(head == null) \{$
 $\text{return } 0;$
 $\}$

$\{$
 $\text{int } x = f(head \rightarrow next);$
 $\text{return } 1 + x;$
 $\}$

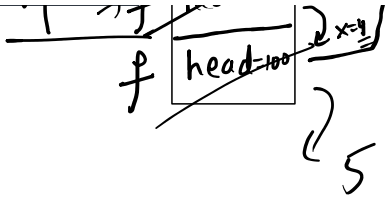


$t(n) = t(n-1) + c \}$ time:
 $t(1) = c$

```
int computeLengthRecursive(ListNode* head) {
    // base case
    if (head == NULL) { // linkedList is empty
        return 0;
    }
    // recursive case
    // f(head) = find the length of the given LL
    // 1. ask your friend to find the length of
    // starts from the node which comes after the
    int x = computeLengthRecursive(head->next);
    return 1 + x;
}
```



time: $O(n)$



time: $O(n)$

space: $O(n)$

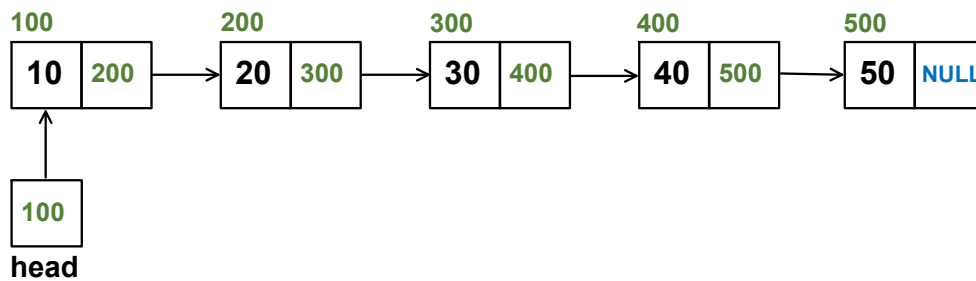
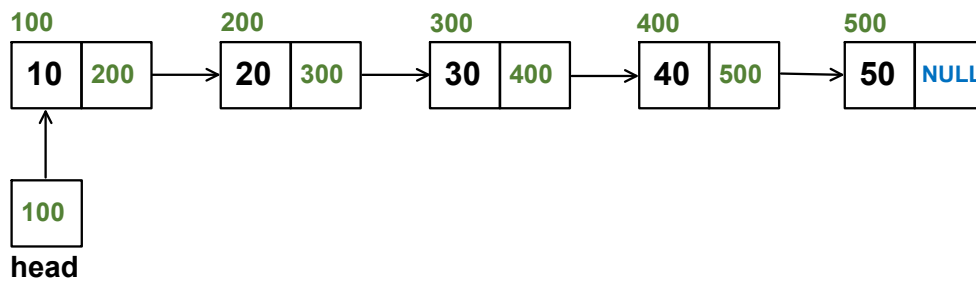
due to
for coll
8HK

i/p: $t=30$
o/p: true

$t=100$
o/p: false

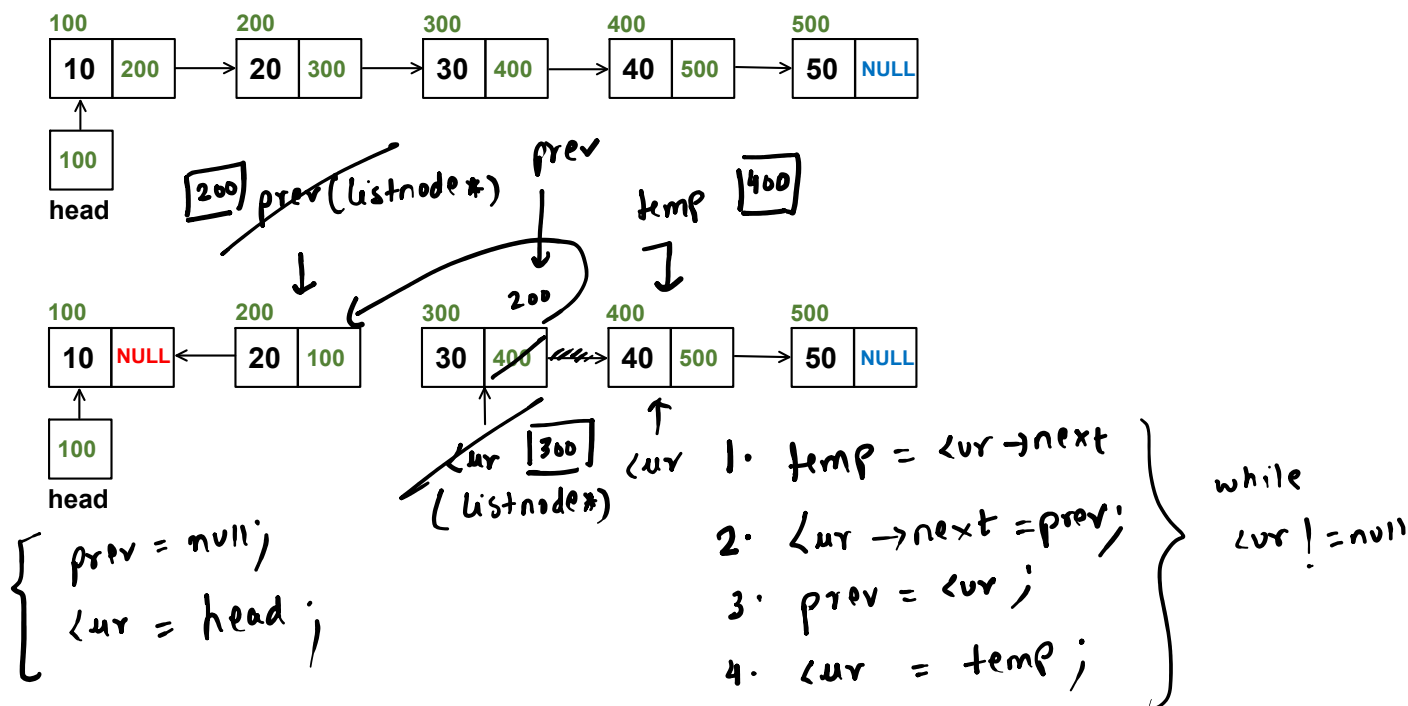
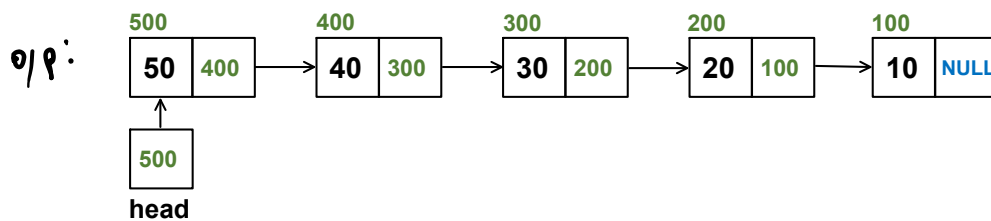
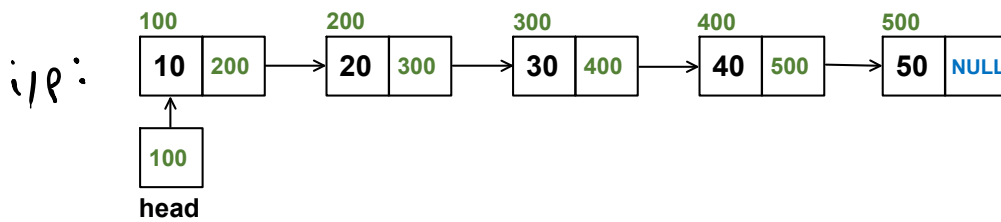
Searching in a Linked List

Given a linked list and a target, check if the target is present in the linked or not.

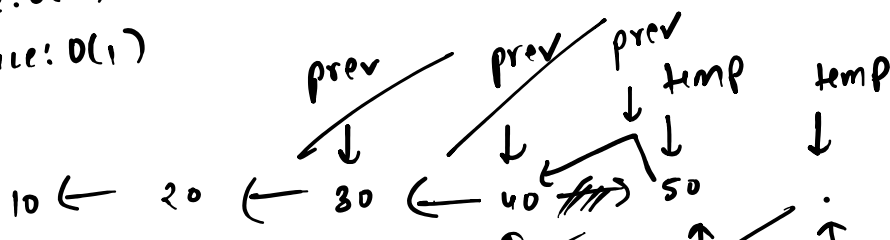


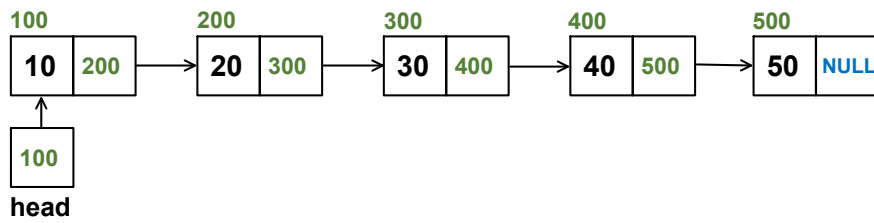
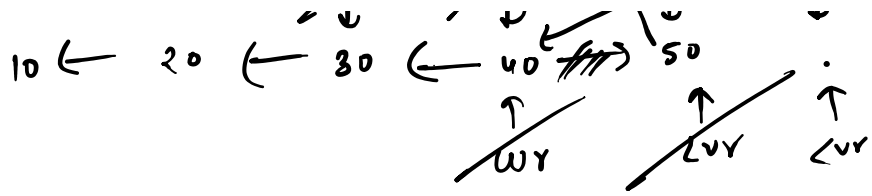
[R] [HW] Write recursive algo. to reverse the linked list

Reverse in a Linked List



time: $O(n)$
space: $O(1)$







$$K = 2$$

K-Reverse in a Linked List

